

VIETNAM AI STARS · 2026

Tally

*Zero-ops real-time features.
A database, not a system.*

Single node with snapshot failover. Install in 60 seconds. Author pipelines with Claude.
Batch is the default today. Streaming is the default in 2030.

Hoang Phan

Founder & Engineering Lead · phan.minhhoang2606@gmail.com · github.com/petrpan26



Zero-ops real-time features.

Real-time features are the engine behind every modern ML product. The difference between catching a fraud transaction and refunding it. Between a feed that engages and one that dies. Between an agent that remembers you and one that feels broken every message. **Today, only FAANG can build them. Tally makes them adoptable on day one.**

Fraud detection

\$6B+/yr

STRIPE RADAR FRAUD PREVENTED (2024)

Sub-100ms scoring on every transaction — velocity, device fingerprint, merchant history, geo signals. Miss the window and the money is gone.

Global card fraud: \$33B/yr market

Real-time personalization

80%

OF NETFLIX VIEWS COME FROM RECS

Rank items as the user scrolls using session signals, dwell time, recent clicks. TikTok's For You Page step-changed engagement over static recommenders.

70%+ of YouTube watch time from recs

LLM agent memory

900M

WEEKLY CHATGPT USERS (2026)

Per-turn context updates — intent history, tool-call state, user preferences. The layer between a coherent agent and one that forgets every message.

Every agent platform needs it

Real-time bidding

\$100B+

ANNUAL PROGRAMMATIC AD MARKET

Sub-50ms bid decisions using audience features, frequency caps, budget pacing. Millions of bids per second, every one powered by real-time feature lookups.

The category that invented real-time features

WHY NOW

Streaming-first is the 2030 default. The tools aren't ready for it.

01

AI-native products can't tolerate batch latency

Every agent, recommender, fraud detector, and LLM context builder shipping in 2026 needs state that's at most seconds stale. Nightly batch is architecturally disqualified.

02

Incremental beats full-table rewrites 3×

I've personally taken a growth-stage marketplace's warehouse bill from \$180K to \$60K by moving transformations to incremental. Batch gets more expensive as data grows. Streaming stays flat.

03

The DevEx penalty for streaming collapsed

Flink SQL used to be harder than dbt SQL. A `@tl.stream` Python class is now easier than a dbt model. The moment streaming is easier than batch, the default flips for every new workload.

04

Claude can author streaming pipelines

Tally's SDK is small enough for an LLM to reason about and declarative enough that a pipeline is a data structure, not a DAG. Only possible because Tally was designed in the Claude Code era.

Every workload an AI-native team writes in 2026 defaults to streaming. The category is flipping. The incumbents weren't built for this moment.

THE PROBLEM

Real-time isn't gated by budget. It's gated by engineering risk.

I have shipped this system three times.

At **Faire** I was the ML platform engineer. We deferred real-time features until the company crossed several hundred million in ARR. I built the internal version — a feature definition library that cut Snowflake spend from \$180K to \$60K and unlocked 3 production deep-learning launches with zero dedicated platform engineers.

At **Fennel AI** (acquired by Databricks, 2025) I re-architected the real-time engine. Five enterprise customers paid for the cluster-shaped version of this exact problem.

At **Viggle** I lead the team building real-time inference infra right now. Saved **\$1M/year** in infrastructure spend in my first six months.

Three companies. Same pain. No one solved it for growth-stage teams. So I'm building it myself.

THE STATUS-QUO STACK

Kafka + Zookeeper	transport	brokers, rebalances
Flink / Spark Streaming	stateful compute	checkpoints, state TTL
Feast / Tecton	feature serving	online/offline sync
Snowflake / BigQuery	warehouse	training data prep
dbt	transformations	full-table rewrites
Airflow / Dagster	orchestration	glue code
2-3 platform engineers	keep it alive	24/7 on-call

6 systems. 3 engineers. \$500K-\$1M / year all-in.

THE SOLUTION

Tally: one binary, one mental model, zero ops.

I

Runs like a database

Single-node Rust database with snapshot failover. No Kafka, Flink, or Zookeeper. Think Redis or Postgres, not a streaming system. Hot-standby replication on the roadmap.

II

One engineer's mental model

Python class in, pipeline out. No DAGs, no operator topology, no checkpoint coordinator. First feature in 10 minutes.

III

Controllable by design

Every feature in code. Every operator explains on a whiteboard. Snapshot on command. Inspect over HTTP. Restart for full recovery.

IV

Not an experiment

Predictable behavior. Bounded memory per key. Documented limits. Adopting Tally is closer to adding a Python library than standing up a data platform.

100× cheaper than Kafka + Flink + feature store on the same workload. That's not why teams adopt it. Teams adopt it because *it isn't a bet*.

ARCHITECTURE

Single-node database. Four layers. Built like Redis.

CLIENT

Python SDK

Declarative `@tl.stream` definitions. Thin TCP client.

Any TCP client

Go, Node, Rust, C++, Java via custom binary protocol.

Claude skills

Author pipelines in prose.
Generate realistic test data.
Parity-test locally — dev box runs the same as prod.

TALLY · SINGLE-NODE DATABASE

TCP protocol

PUSH / GET / SET / MSET / REGISTER over persistent length-prefixed frames.

Pipeline engine

Operators (count, sum, avg, min, max, distinct_count, last). Expression AST. Cross-stream views. Lookups.

In-memory state

Everything stays in RAM. Sub-microsecond reads. Bounded memory per key.

Event log + snapshot persistence

Shipped: periodic snapshots. Phase 9 (Q2 2026): incremental snapshots + durable event log for training-data replay.

OUT OF BAND

Web debug UI

Inspect any key's state. Watch live events. Step through operators visually.

HTTP management

Register pipelines, /metrics, /health, debug inspection.

Local disk snapshot

Versioned binary file. ~5s recovery per 1M keys.

No distributed consensus. No checkpoint coordinator. No rebalance protocol. Restart = full recovery in seconds.

A feature is a Python attribute. A pipeline is a class. Claude writes them.

```
import tally as tl

@tl.stream(key="user_id")
class Transactions:
    count_1h = tl.count(window="1h")
    sum_1h   = tl.sum("amount", window="1h")
    avg      = tl.derive("sum_1h / count_1h")

f = app.push(Transactions, {"user_id": "u_1", "amount": 50})
f.avg    # 48.2
```

Why this matters

- Feature definitions live next to application code — not a separate YAML DSL or UI.
- Same API a solo founder and a Series B ML team both pick up in 10 minutes.
- Web debug UI ships with Tally: inspect any key's state, watch live events, step through operators.
- Claude skills author pipelines AND generate realistic test data for parity testing. Dev box runs the same as prod because it's one node.

This is the abstraction incumbents *cannot retrofit*. Their SDKs are too large for an LLM to hold in context. Ours is small by design.

Four locks that compound, not four features that copy.

1 Architectural lock-in

Materialize, RisingWave, Flink, ksqlDB assume distributed state from day one. Shipping a single binary isn't a flag — it's a two-year rewrite of their state layer.

2 Price invisibility

Databricks sales teams don't knock on seed-stage doors for a \$500/mo deal. Tally at \$0 OSS and \$20-\$5K/mo cloud is beneath every incumbent's go-to-market.

3 Features-first DX

Python class in, running pipeline out. No SQL views, no DAG config, no feature store YAML. The only player close on this axis is Chalk — enterprise-only on Kubernetes.

4 AI-agent native by construction

Claude skills author pipelines, generate test data, and debug live state. An MCP server exposes the running node to any AI agent. The SDK is small by design so an LLM can hold it all in context.

Copying one lock doesn't threaten us. Copying all four means rebuilding the company.

Same throughput as a Flink cluster. Zero platform engineers.

100K+

events/sec sustained on one thread

<100μs

PUSH latency (p99, single event)

<50μs

GET latency (p99)

<5KB

memory per key, 10 mixed features

SINGLE-NODE CAPACITY · R6G MEMORY-OPTIMIZED **r6g.xlarge** · 4c/32 GB → 5M entities · \$150/mo • **r6g.4xlarge** · 16c/128 GB → 20M entities · \$600/mo •

r6g.16xlarge · 64c/512 GB → 80M entities · \$2.4K/mo

Throughput: **100K evt/s** on v1 (single-threaded, Redis-style). Key-partitioned multi-threading in v2 scales to **500K+ evt/s** on the same hardware — no code changes, just a config flag.

STATUS QUO · KAFKA + FLINK + FEAST

- 4 systems to operate
- 3 dedicated platform engineers
- Cloud infrastructure: \$15K–\$40K / month
- Salaries: \$400K–\$600K / year

~\$450K–\$1M / year

WITH TALLY

- Single-node DB, 0 dedicated engineers
- r6g memory-optimized: \$150–\$2,400 / month
- Snapshot storage: cents / month
- Platform team: \$0

~\$2K–\$30K / year

The category consolidated upward in 2025.

PLAYER	SINGLE-NODE DB?	SELF-SERVE PRICE?
Kafka + Flink (Confluent)	no — cluster	enterprise
Materialize	no — managed	mid-market
RisingWave	no — cluster	enterprise sales
Tinybird	managed only	no low tier
Chalk	no — Kubernetes	contact sales
Feast	BYO Kafka + Redis	free, self-assembled
Tecton	absorbed	Databricks, Aug 2025
Fennel	absorbed	Databricks, Apr 2025
Neon	absorbed	Databricks \$1B, 2025
Tally	yes — single node	\$0 – \$5K/mo

Since 2023, Databricks has spent over **\$4B** acquiring MosaicML (\$1.3B, 2023), Tabular (~\$2B, 2024), Fennel (Apr 2025), Neon (\$1B, May 2025), and Tecton (Aug 2025). They rolled it into one enterprise platform called *Agent Bricks*.

The real-time feature store category effectively no longer exists as independent startups. Confluent owns managed Flink. Databricks owns the feature store layer. Neither sells to seed-stage founders.

The wedge stays open until an incumbent rewrites their cluster-shaped architecture into a single binary — a multi-year project none of them will start.

I was at Fennel through the Databricks acquisition. I saw this consolidation happen from inside the space.

Core engine is real. Phase 9 (incremental snapshots) substantially shipped.

SHIPPED · CORE STREAMING FUNCTIONALITY

Real-time aggregations

Count, sum, avg, min, max over sliding windows — updated on every event.

Approximate distinct counts

Unique merchants, IPs, devices at any cardinality with bounded memory.

Derived expressions

Compute features from other features: ratios, velocity spikes, composite scores.

Cross-stream features

Reference features from other streams. Lookup by alternate keys.

One round-trip push

Event in, updated features out in the same call. Sub-millisecond.

Snapshot recovery

Periodic state dumps. Restart equals full recovery in seconds.

Incremental snapshots (Phase 9)

Base + delta format v6. Dirty/deleted tracking. Durable event log. Recovery tested.

Python SDK & Claude skills

Declarative @tl.stream classes. Claude authors pipelines from prose.

NEXT · COMPETITION TIMELINE

Q2 2026 Design partner outreach: 5 ML leads from Fennel + Viggle network

Q2 2026 Benchmark harness + public performance report

Q2 2026 MCP server + web debug UI

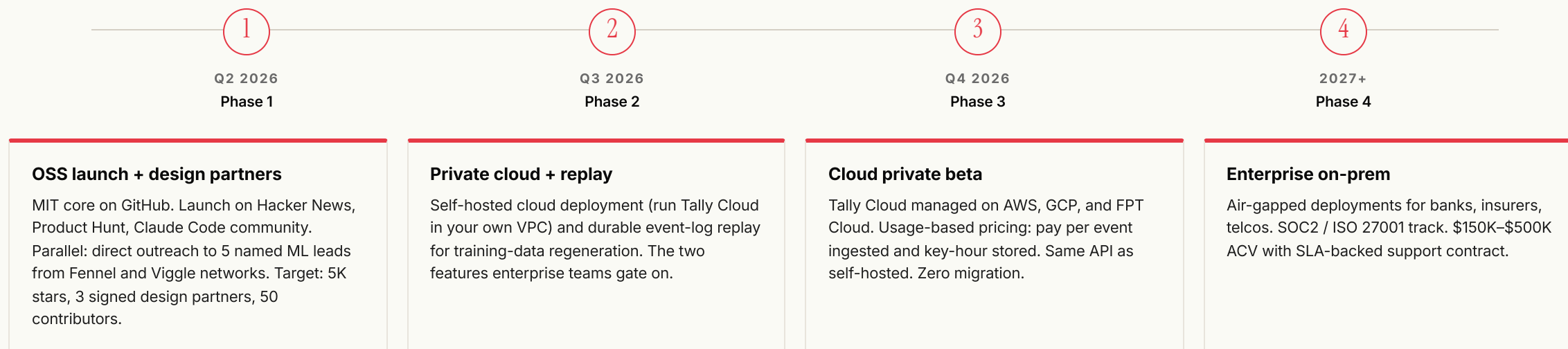
Q3 2026 Open source launch on HN, Product Hunt, Vietnamese docs

Q4 2026 Tally Cloud private beta on FPT Cloud + AWS

Q4 2026 10 production deployments milestone

No external data. No ML model to train. Deploys on any Linux box. Founder has built this class of system twice before.

OSS → usage-based cloud → enterprise. The Vercel playbook.



DevEx wins new workloads. New workloads compound into platform dependency. Platform dependency converts to revenue. Vercel (**\$9.3B**), Supabase (**\$5B**), PlanetScale (**\$1B+**) all built this way.

TEAM

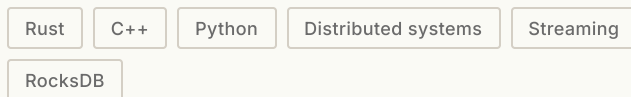
I've shipped this system three times. Now I'm building the version I wished existed.



Hoang Phan

Founder & Engineering Lead

University of Waterloo
BCS, Minor in Finance



Viggle AI · Engineering Lead, Toronto · 2025–present

Leading the 4-engineer team building Viggle's real-time video inference platform. Cut time-to-first-chunk from 8s to 2s. Saved \$1M/year in infrastructure. Scaled Toronto engineering from 1 to 8. Shipped the platform foundation underneath 7 signed customers.

Fennel AI · Distributed Systems Engineer · 2024–2025 (acquired by Databricks)

Re-architected core streaming join and window operators for 10× runtime and memory efficiency. Designed percentile data structures that landed 5 enterprise customers including Upwork and Rippling. Co-owned infrastructure architecture with the CTO. **I built the class of system Tally is, at the company Databricks absorbed.**

Faire · ML Platform Engineer · 2022–2023

Shifted Faire from XGBoost-only to deep-learning-capable with 3 production launches and zero dedicated deployment engineers. Built the internal feature definition library Tally commercializes. Cut Snowflake spend from \$180K to \$60K via incremental transformations.

Google SWE Intern, 2021 · APIO Silver Medal, 2nd in Vietnam

Placed 35th of 164 across 30 countries at the Asia-Pacific Informatics Olympiad. A decade of systems and competitive programming foundation.

Usage-based. The Supabase playbook, one layer up.

ADOPTION ENGINE

Open Source

MIT core, self-host

\$0

- Full server + Python SDK
- Single-node deploy with failover
- Community support, Discord + GitHub
- For hobbyists, indie hackers, OSS projects

PRIMARY REVENUE

Cloud

Managed, usage-based

\$20 – \$5K / mo

- Free tier for hobbyists
- Pay per event ingested + key-hour stored
- Automated snapshots, dashboards, alerts
- Same API as self-hosted — zero migration

HIGHEST ACV

Enterprise

On-prem + support

\$150K – \$500K / yr

- Air-gapped deployment
- SLA-backed support contract
- SOC2 / ISO 27001 assistance
- For banks, insurers, telcos

HORIZON	TARGET ARR	REAL COMPARABLE AT SAME STAGE
Year 3	\$10–20M	Supabase at year 3 (~\$10M ARR)
Year 5	\$60–80M	Supabase at year 5 · \$70M ARR · \$5B val · 250% YoY
Year 10	\$800M–\$1.5B	Vercel, MongoDB, dbt Labs class
Year 15	\$3–5B	Snowflake / Databricks class · \$10B+ valuation

Supabase is \$5B on \$70M ARR today, targeting \$10B. Tally is one layer up the stack. Databricks spent **\$4B+** consolidating the enterprise tier of our category since 2023.



THE VISION

Zero-ops. Streaming-first. Made in Vietnam.

Tally is the default streaming data primitive for the next generation of AI-native products. Every workload an AI-native team writes in 2026 defaults through it. Built in Vietnam. Used everywhere.

WHAT WE ARE ASKING FOR

Mentorship

Guidance from VAS judges and mentors on taking a Vietnam-origin developer-tools startup to a global audience — especially on the OSS → managed-cloud → enterprise sequence.

Design partners

Warm intros to 3–5 growth-stage fintech, e-commerce, or AI startups — Vietnamese or global — willing to be first production users in exchange for direct engineering support and a case study.

Seed capital

\$1M pre-seed. **First hire: GTM co-founder.** Second hire: senior Rust engineer. Funds 18 months runway, Tally Cloud beta on FPT Cloud + AWS, and 10 design-partner deployments. I know my blind spot is sales motion, not engineering.

Cảm ơn. Thank you.

phan.minhhoang2606@gmail.com · github.com/petrpan26